

Operators & Expressions

operators



- Operators are used to perform operations on variables and values.
- Python divides the operators in the following groups:

Arithmetic operators

Assignment operators

Comparison operators

Logical operators

Identity operators

Membership operators

Bitwise operators

Python Arithmetic Operators

- Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Meaning	Example
+	Add two operands or unary plus	$x + y + 2$
-	Subtract right operand from the left or unary minus	$x - y - 2$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

Assignment operators



- Assignment operators are used in Python to assign values to variables.
- $a = 5$ is a simple assignment operator
- There are various compound operators in Python like $a += 5$ that adds to the variable and later assigns the same.
- It is equivalent to $a = a + 5$.

Assignment operators

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
=	x = 5	x = x 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Comparison operators



Comparison operators are used to compare values.

It returns either **True** or **False** according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	$x > y$
<	Less than - True if left operand is less than the right	$x < y$
==	Equal to - True if both operands are equal	$x == y$
!=	Not equal to - True if operands are not equal	$x != y$
>=	Greater than or equal to - True if left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to - True if left operand is less than or equal to the right	$x <= y$

Logical operators



Logical operators are used to combine conditional statements:

Logical operators are the **and**, **or**, **not** operators.

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

Logical operators



Example

a=10

b=20

if a>1 and b<25:

 print("correct")

else:

 print("Wrong")

Identity Operators



- They are used to check if two values (or variables) are located on the same part of the memory.
- Two variables that are equal does not imply that they are identical.
- **is** and **is not** are the identity operators in Python.

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Identity Operators

```
x1 = 5  
y1 = 5  
x2 = 'Hello'  
y2 = 'Hello'  
x3 = [1,2,3]
```

```
# Output: True  
print(x1 is y1)
```

```
# Output: True  
print(x2 is y2)
```

```
# Output: False  
print(x1 is x3)
```

```
# Output: True  
print(x1 is not x3)
```

Membership Operators



They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).

in and **not in** are the membership operators in Python.

In a dictionary we can only test for presence of **key**, not the **value**.

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Membership Operators



```
x = 'Hello world'
y = {1 : 'a', 2 : 'b'}
```

Output: True

```
print('H' in x)
```

Output: True

```
print('hello' not in x)
```

Output: True

```
print(1 in y)
```

Output: False

```
print('a' in y)
```

Bitwise Operators



Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

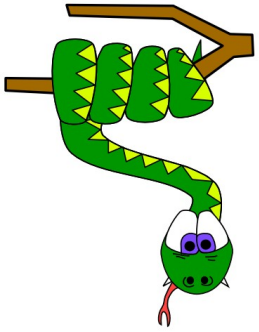
For example, 2 is 10 in binary and 7 is 111.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Bitwise Operators

In the table below: Let $x = 10$ (0000 1010 in binary) and $y = 4$ (0000 0100 in binary)

Operator	Meaning	Example
&	Bitwise AND	$x \& y = 0$ (0000 0000)
	Bitwise OR	$x y = 14$ (0000 1110)
~	Bitwise NOT	$\sim x = -11$ (1111 0101)
^	Bitwise XOR	$x \wedge y = 14$ (0000 1110)
>>	Bitwise right shift	$x \gg 2 = 2$ (0000 0010)
<<	Bitwise left shift	$x \ll 2 = 40$ (0010 1000)



Precedence and Associativity of Operators

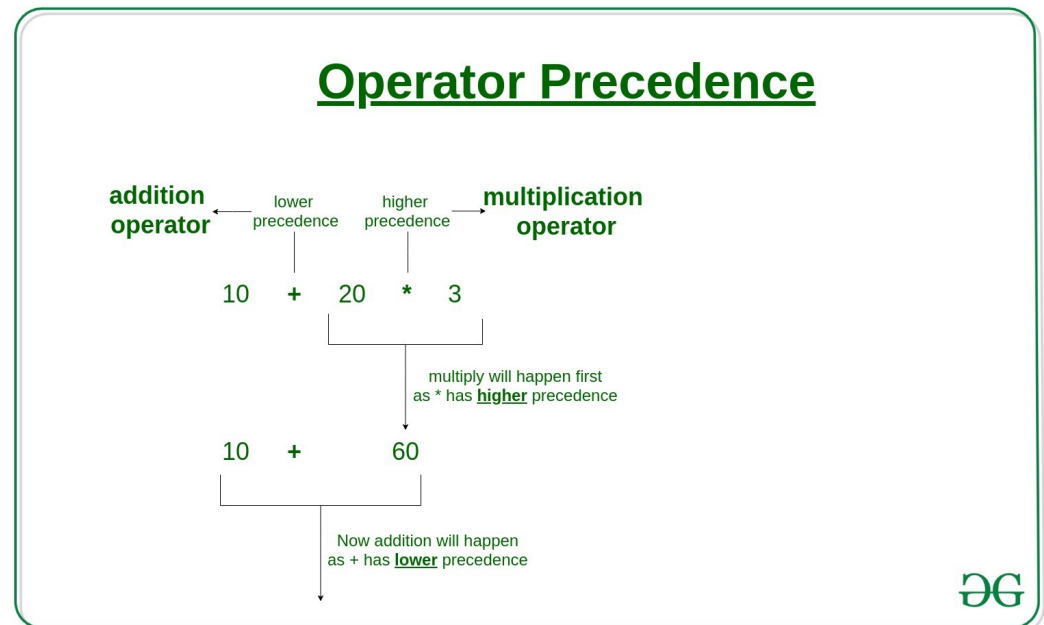
Precedence and Associativity



- When dealing with operators in Python we have to know about the concept of Python operator precedence and associativity as these determine the priorities of the operator otherwise, we'll see unexpected outputs.
- **Operator Precedence:** This is used in an expression with more than one operator with different precedence to determine which operation to perform first.

Example: Solve

$$10 + 20 * 3$$



Precedence and Associativity



OPERATOR	DESCRIPTION	ASSOCIATIVITY
()	Parentheses	left-to-right
**	Exponent	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <=	Relational less than/less than or equal to	left-to-right
> >=	Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
is, is not	Identity	left-to-right
in, not in	Membership operators	left-to-right
&	Bitwise AND	left-to-right

Precedence and Associativity



<code>^</code>	Bitwise exclusive OR	left-to-right
<code> </code>	Bitwise inclusive OR	left-to-right
<code>not</code>	Logical NOT	right-to-left
<code>and</code>	Logical AND	left-to-right
<code>or</code>	Logical OR	left-to-right
<code>=</code>	Assignment	right-to-left
<code>+= -=</code>	Addition/subtraction assignment	
<code>*= /=</code>	Multiplication/division assignment	
<code>%= &=</code>	Modulus/bitwise AND assignment	
<code>^= =</code>	Bitwise exclusive/inclusive OR assignment	
<code><<= >>=</code>	Bitwise shift left/right assignment	

Precedence and Associativity



```
# Precedence of or & and
meal = "fruit"
money = 10
if meal == "fruit" or meal == "sandwich" and
money >= 2:
    print("Lunch being delivered")
else:
    print("Can't deliver lunch")
```

OUTPUT :

Lunch being delivered

Precedence and Associativity



Associativity of Python Operators

- We can see in the above table that more than one operator exists in the same group. These operators have the same precedence.
- When two operators have the same precedence, associativity helps to determine the order of operations.
- Associativity is the order in which an expression is evaluated that has multiple operators of the same precedence.
- Almost all the operators have left-to-right associativity.

Precedence and Associativity



```
# Left-right associativity
# Output: 3
print(5 * 2 // 3)

# Shows left-right associativity
# Output: 0
print(5 * (2 // 3))
```

Exponent operator `**` has right-to-left associativity in Python.

```
# Shows the right-left associativity of **
# Output: 512, Since  $2^{(3^2)} = 2^9$ 
print(2 ** 3 ** 2)

# If 2 needs to be exponentated first, need to use ()
# Output: 64
print((2 ** 3) ** 2)
```